AUTONOMOUS PID-CONTROLLED LINE-FOLLOWING BOT

ABSTRACT

This project presents the design, implementation, and evaluation of an autonomous line-following robotic system driven by an ESP32 microcontroller, a 5-channel infrared (IR) reflectance sensor array, and a TB6612FNG dual motor driver. The system employs a closed-loop Proportional–Integral–Derivative (PID) control algorithm to continuously compute positional error from the sensor array and dynamically regulate differential motor speeds via Pulse Width Modulation (PWM). Advanced firmware routines, including gap-tolerance filtering and dynamic line-recovery pivoting, ensure smooth trajectory tracking and stability during track discontinuities or severe overshoot conditions. Hardware power management utilizes isolated logic and drive rails to mitigate electromagnetic interference and voltage drops. The mechanical frame features a custom-engineered chassis developed in Fusion 360. Practical applications demonstrate robust human-machine interaction, real-time embedded control, and sensor fusion in autonomous mobile robotics.

LIST OF FIGURES

LIST OF ABBREVIATIONS

Abbreviation	Full Form
IoT	Internet of Things
LED	Light Emitting Diode
IR	Infrared
MCU	Micro-Controller Unit
PID	Proportional–Integral–Derivative
PWM	Pulse Width Modulation
GPIO	General Purpose Input/Output
DC	Direct Current
LDO	Low Dropout Regulator
RPM	Revolutions Per Minute

NOTATION

Notation	Physical / Electrical Quantity
V	Volts (Voltage)
mV	Millivolts
A	Amperes (Current)
mA	Milliamperes
Ω	Ohms (Resistance)
MHz	Megahertz (Frequency)
ms	Milliseconds (Time)

1. INTRODUCTION

1.1. Background and Motivation

Autonomous mobile robots rely heavily on precise sensor feedback and real-time control algorithms to navigate structured environments. Line-following robots represent a foundational benchmark in control systems engineering. Traditional line followers rely on basic threshold-based (bang-bang) control, leading to oscillations, jerky movements, and frequent line loss.

This project addresses these limitations by integrating a multi-channel analog sensor array with a closed-loop PID control algorithm. Using an ESP32 microcontroller, the system computes a weighted continuous error value to execute smooth, adaptive path tracking over varying curvatures and line breaks.

1.2. System Overview

The physical platform utilizes a 5-channel IR reflectance tracker module mounted at the front chassis. The analog voltage output from each sensor is acquired by the ESP32's Analog-to-Digital Converters (ADC). The system converts these signals into continuous positional error values, which are processed through digital PID filtering to calculate differential drive correction signals. A TB6612FNG dual H-bridge motor driver receives these PWM signals to control two high-torque N20 gear motors.

1.3. Power Architecture and Connectivity

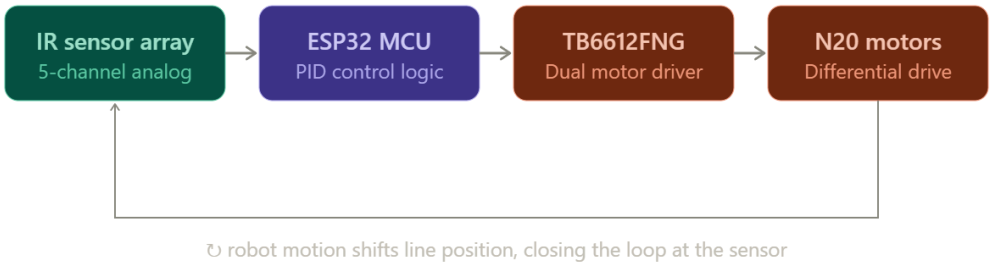
To ensure system stability, the power architecture separates logic power from motor drive power. High-current transients caused by motor direction switching can cause brown-outs on microcontroller rails. The primary supply is an 11.1 V (3S) Li-ion battery pack powering the motors directly via the TB6612FNG, while a discrete LM2596 buck converter steps down the supply voltage to power the ESP32 and sensor arrays safely.

1.4. Workflow and Engineering Track

The development lifecycle was divided into three modular technical tracks:

1. **Embedded Hardware Track:** Circuit schematic design, power domain isolation, motor driver interfacing, and chassis fabrication.
2. **Firmware Track:** Real-time C++ control logic, multi-channel ADC acquisition, PID algorithm implementation, and fault-recovery routines.
3. **Control & Mechanical Track:** CAD modeling in Fusion 360, PID tuning parameter optimization (K_p, K_i, K_d), and track response benchmarking.

2. SYSTEM ARCHITECTURE



Data Flow Pipeline:

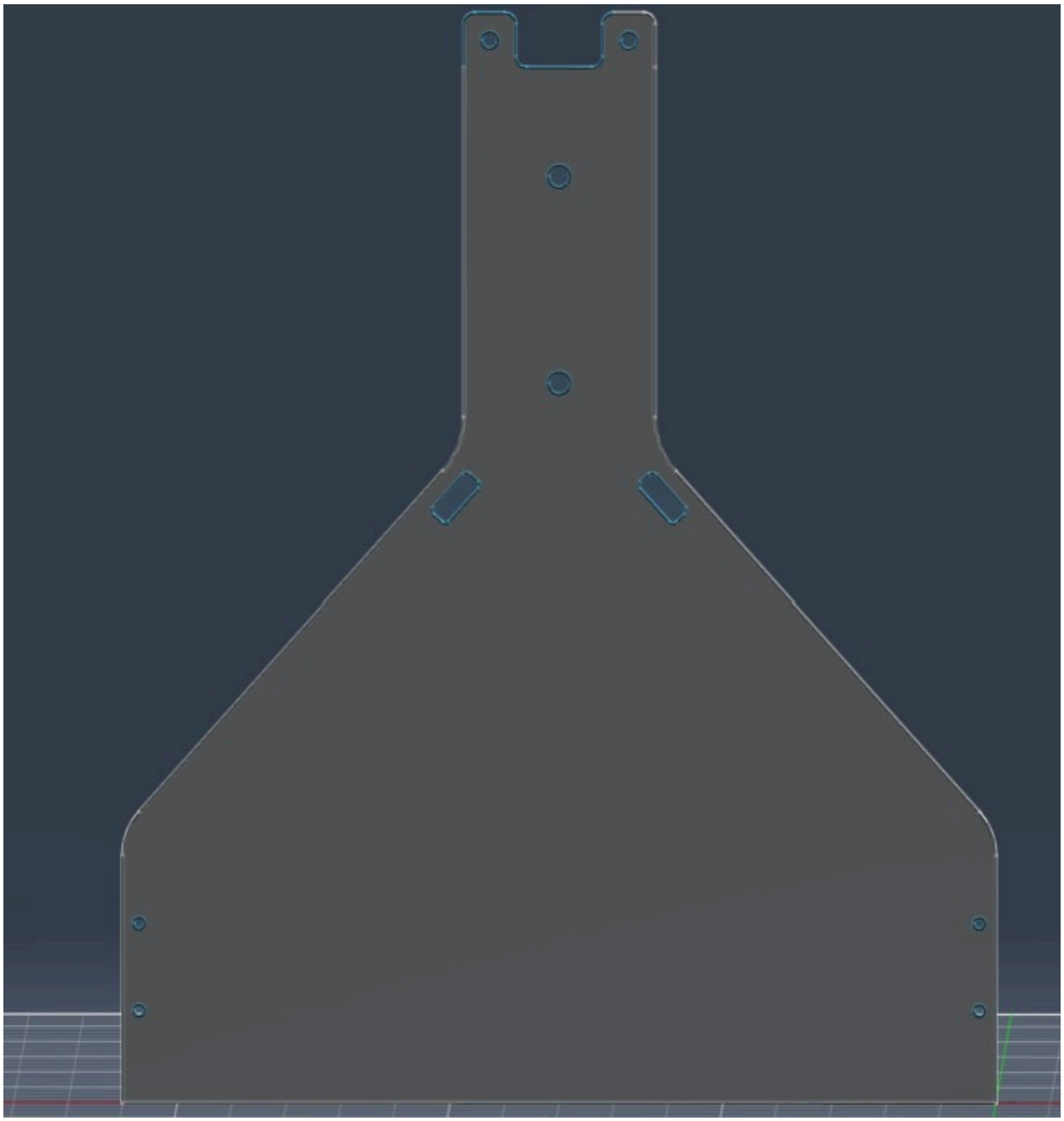
1. **Acquisition:** The 5-channel IR array samples track reflectance levels continuously.
2. **Binarization:** Each analog channel is thresholded into binary states (0 for light surface, 1 for dark track).
3. **Positional Weighting:** Spatial weights $\{-2, -1, 0, 1, 2\}$ translate binary array states into a single scalar error parameter (E).
4. **PID Calculation:** Digital PID terms derive a correction signal (U) from the calculated error.
5. **PWM Signal Generation:** Motor drive signals (PWM_{Left}, PWM_{Right}) are calculated using baseSpeed $\pm U$.
6. **Execution:** The TB6612FNG modifies current delivery to the N20 motors to adjust the robot's heading.

3. HARDWARE SPECIFICATIONS AND BILL OF MATERIALS

3.1. Required Components

Item	Component Description	Operating Parameters	Quantity
1	ESP32 Development Board	240 MHz Dual-Core, 3.3 V Logic	1
2	Waveshare 5-Channel IR Tracker	5-Channel Reflectance Array	1
3	TB6612FNG Dual Motor Driver	Max $1.2\ \text{A}$ continuous per channel	1
4	N20 Micro Gear Motors	Rated $12\ \text{V DC}$, $600\ \text{RPM}$	2
5	18650 Li-ion Cells	$3.7\ \text{V}$, $2200\ \text{mAh}$ per cell (Series 3S)	3
6	LM2596 Step-Down Buck Converter	Adjustable DC-DC, switched output $5\ \text{V}$	1
7	Custom Frame Assembly	Fusion 360 Custom CAD Design	1

CUSTOM CHASIS (FUSION 360:



4. PIN AND HARDWARE INTERFACING

4.1. IR Sensor Interfacing to ESP32

Sensor Channel	Position Weight	Function / Signal	ESP32 Pin
Far Left (L_2)	-2	ir1	GPIO 27
Left (L_1)	-1	ir2	GPIO 34
Center (C)	0	ir3	GPIO 36 (VP)
Right (R_1)	$+1$	ir4	GPIO 39 (VN)
Far Right (R_2)	$+2$	ir5	GPIO 35

4.2. Motor Driver Interfacing to ESP32

ESP32 Pin	TB6612FNG Pin	Functional Description
GPIO 13	PWMA	Left Motor Speed Control (PWM Output)
GPIO 14	PWMB	Right Motor Speed Control (PWM Output)
GPIO 25	IN1	Left Motor Direction Input Bit 1
GPIO 26	IN2	Left Motor Direction Input Bit 2
GPIO 32	IN3	Right Motor Direction Input Bit 1
GPIO 33	IN4	Right Motor Direction Input Bit 2

GND	GND	Common Logic and Power Ground System

5. POWER CALCULATIONS AND MANAGEMENT

5.1. Voltage Distribution Analysis

- **Primary Battery Pack Supply (V_{BAT}):**

$$V_{BAT} = 3 \times 3.7 \text{ V} = 11.1 \text{ V nominal}$$

- **Motor Driver Motor Rail (V_M):** Directly connected to V_{BAT} (11.1 V), matching the operating range of the 12 V rated N20 motors.
- **Microcontroller and Sensor Rail (V_{CC}):** Stepped down via the LM2596 buck converter to 5.0 V , then regulated to 3.3 V by the ESP32's onboard LDO.

5.2. Current and Power Calculations

- **N20 Motor Current Draw:**
  - Stall Current (I_{stall}): ~ $\approx 0.6 \text{ A}$ per motor at 11.1 V .
  - Operational Drive Current (I_{run}): ~ $\approx 0.12 \text{ A}$ per motor.
  - Total Peak Drive Current ($I_{motor,peak}$):

$$I_{motor,peak} = 2 \times 0.6 \text{ A} = 1.20 \text{ A}$$

- **Logic Current Draw:**
  - ESP32 Operating Current (I_{ESP32}): ~ $\approx 160 \text{ mA}$ (active processing).
  - IR Sensor Array Draw (I_{IR}): ~ $\approx 50 \text{ mA}$.
  - Total Logic Current (I_{logic}):

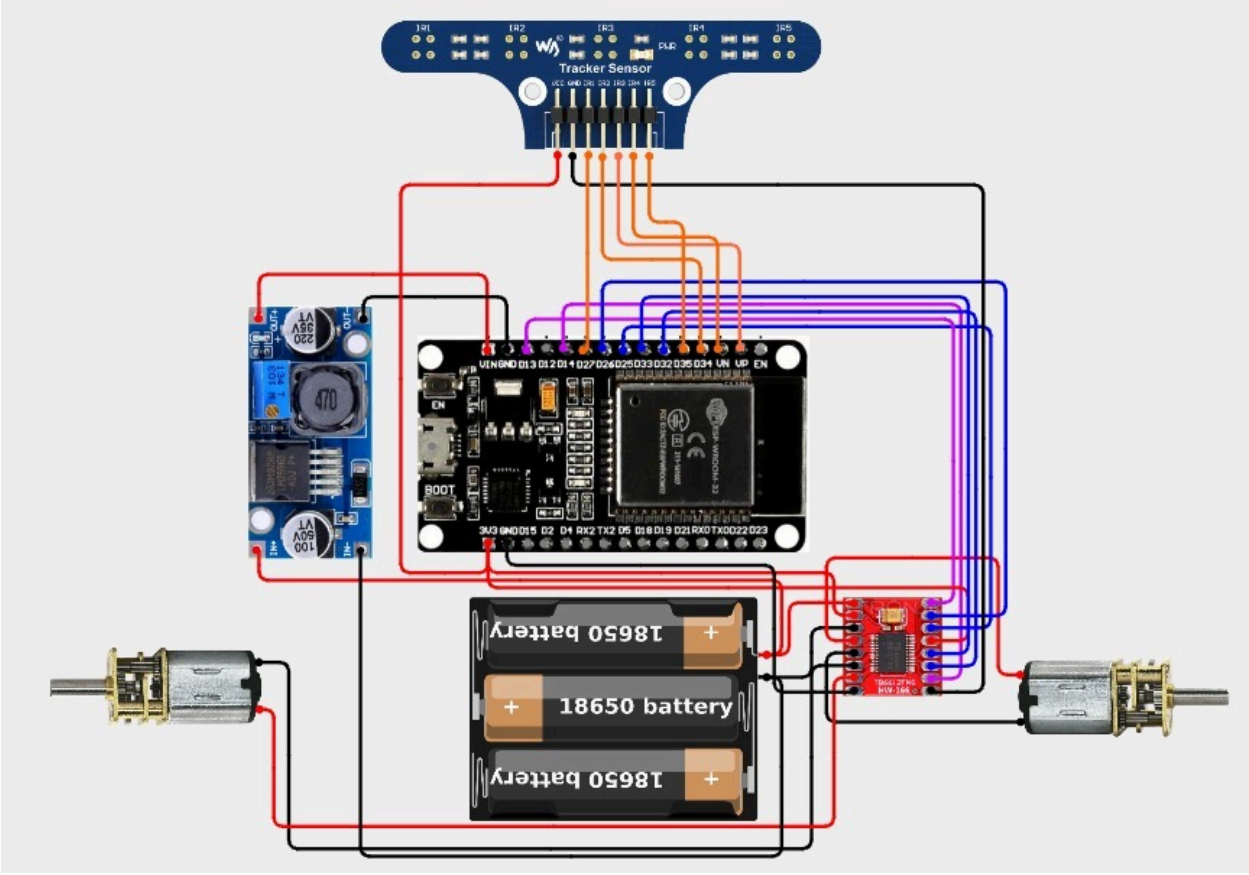
$$I_{logic} = 160 \text{ mA} + 50 \text{ mA} = 210 \text{ mA} = 0.21 \text{ A}$$

- **Total Dynamic Peak Load:**

$$I_{total,peak} = I_{motor,peak} + I_{logic} = 1.20\ \text{A} + 0.21\ \text{A} = 1.41\ \text{A}$$

Conclusion: The selected 3S Li-ion battery configuration, combined with an LM2596 buck converter, safely satisfies the system's operational current requirements without risking MCU power resets.

COMPLETE CIRCUIT DIAGRAM



6. FIRMWARE AND CONTROL ALGORITHM IMPLEMENTATION

6.1. Control Mathematics

1. **Weighted Centroid Position Error Calculation:**

   Given sensor binary states $S_i \in \{0, 1\}$ and positional weights $W_i \in \{-2, -1, 0, 1, 2\}$ for $i \in \{1..5\}$:

$$E = \frac{\sum_{i=1}^{5}(S_i \times W_i)}{\sum_{i=1}^{5} S_i}$$

2. **Discrete PID Algorithm:**

$$u(t) = K_p \cdot E(t) + K_i \int_0^t E(\tau)d\tau + K_d \frac{dE(t)}{dt}$$

   In software discretization:

$$\text{Integral: } I_k = I_{k-1} + E_k$$

$$\text{Derivative: } D_k = E_k - E_{k-1}$$

$$\text{Correction: } U_k = (K_p \times E_k) + (K_i \times I_k) + (K_d \times D_k)$$

3. **Motor Speed Output Assignment:**

$$\text{Left Motor PWM } (PWM_L) = \text{ Clamp}(\text{baseSpeed} + U_k, 0, 255)$$

$$\text{Right Motor PWM } (PWM_R) = \text{ Clamp}(\text{baseSpeed} - U_k, 0, 255)$$

6.2. Firmware Source Code

```
//defining pins for IR sensors
#define ir1 27
#define ir2 34
```

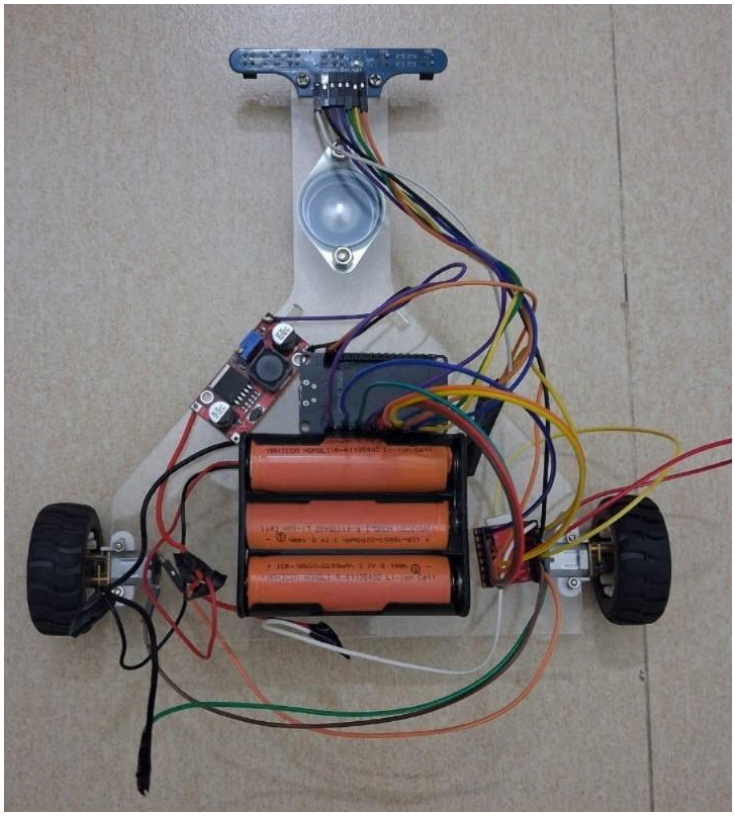
```
#define ir3 36 //SensorVP -> VP
#define ir4 39 //SensorVN -> VN
#define ir5 35

//defining pins for motor driver
#define PWMA 13
#define PWMB 14
#define IN1 25
#define IN2 26
#define IN3 32
#define IN4 33

//weights for each IR sensor
const int weights[5] = {-2,-1,0,1,2};

//Proportionality constant
const float Kp = 18;
//Derivative constant
const float Kd = 7;
//Integral constant
const float Ki = 0.2;

//default speed for bot
const float baseSpeed = 110;
const float pivotSpeed = 100;
const float minSpeed = 0;

float previousError = 0;
float integralError = 0;
const float integralLimit = 50;

//for small gaps in between like -> "---"
unsigned long lostLineTime = 0;
const int gapTolerance = 50; //50 ms

void setMotors(float leftSpeed,float rightSpeed){
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, HIGH);
  digitalWrite(IN4, LOW);

  ledcWrite(PWMA, constrain(leftSpeed, minSpeed, 255));
  ledcWrite(PWMB, constrain(rightSpeed, minSpeed, 255));
}
```

```
void setup() {
  Serial.begin(115200);
  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
  pinMode(IN3, OUTPUT);
  pinMode(IN4, OUTPUT);
  //pinMode(STBY, OUTPUT);

  pinMode(ir1, INPUT);
  pinMode(ir2, INPUT);
  pinMode(ir3, INPUT);
  pinMode(ir4, INPUT);
  pinMode(ir5, INPUT);

  //setting up the frequency and resolution of the PWM
  ledcAttach(PWMA, 1000, 8);
  ledcAttach(PWMB, 1000, 8);
  //digitalWrite(STBY, HIGH);
}

void loop() {
  int l2 = analogRead(ir1);
  int l1 = analogRead(ir2);
  int c = analogRead(ir3);
  int r1 = analogRead(ir4);
  int r2 = analogRead(ir5);

  int readings[5] = {l2,l1,c,r1,r2};
  int binary[5];
  int multi[5];

  float multiSum = 0;
  float sum = 0;
  float error = 0;

  //converting the values to 0 or 1
  for(int i=0;i<5;i++){
    //threshold value is set as 1700 considering all readings from calibrated
values!
    if(readings[i] > 1700){
      binary[i] = 0; //WHITE -> 0
    }
    else{
      binary[i] = 1; //BLACK -> 1
    }
```

```
    multi[i] = binary[i] * weights[i];
    multiSum += multi[i];
    sum += binary[i];
  }
  //if all sensors detect WHITE then, division by zero will occur -> gotta prevent
that!
  if(sum !=0){
    lostLineTime = 0;
    error = multiSum / sum; //error tells how far the bot is away from the centre
    integralError += error;
    integralError = constrain(integralError, -integralLimit, integralLimit);

    float derivative = error - previousError;
    float crct = Kp*error + Ki*integralError + Kd*derivative;

    previousError = error;

    float lsp = baseSpeed + crct;
    float rsp = baseSpeed - crct;
    setMotors(lsp, rsp);
  }
  else{
    unsigned long now = millis();
    if(lostLineTime == 0){
      lostLineTime = now;
    }
    unsigned long lostDuration = now - lostLineTime;
    if(lostDuration < gapTolerance){
      setMotors(baseSpeed, baseSpeed);
    }
    else{
      integralError = 0;
      if(previousError >= 0){
        digitalWrite(IN1, HIGH);
        digitalWrite(IN2, LOW);
        digitalWrite(IN3, LOW);
        digitalWrite(IN4, HIGH);
        ledcWrite(PWMA, pivotSpeed);
        ledcWrite(PWMB, pivotSpeed);
      }
      if(previousError < 0){
        digitalWrite(IN1, LOW);
        digitalWrite(IN2, HIGH);
        digitalWrite(IN3, HIGH);
        digitalWrite(IN4, LOW);
```

```
      ledcWrite(PWMA, pivotSpeed);
      ledcWrite(PWMB, pivotSpeed);
    }
   }
  }
  //Serial.println("Error: "+String(error));
  Serial.println("Sensor Readings: "+String(l2)+" "+String(l1)+" "+String(c)+"
"+String(r1)+" "+String(r2));
  Serial.println("Binary readings:");
  for(int i=0;i<5;i++){
   Serial.print(String(binary[i])+ " ");
  }
  Serial.println("");
}
```

7. RESULTS AND DISCUSSION



7.1. Experimental Performance Metrics

The system was evaluated across multiple track geometries with varying turn radii ($10\ \text{cm}$ to $50\ \text{cm}$) and track gaps ($10\ \text{mm}$ to $40\ \text{mm}$).

- **Continuous Path Tracking:** The closed-loop PID controller suppressed oscillations on

straight tracks and minimized overshoot on curved segments.

- **Gap Navigation Response:** The 50 ms inertia-driven gap tolerance allowed the bot to traverse track disruptions up to $35\ \text{mm}$ in length without veering off course.
- **Line Recovery Validation:** When overshooting sharp 90° turns, the bot successfully pivoted in the direction of the last recorded error vector until line acquisition was restored.

8. CONCLUSION

The design and hardware implementation of the Autonomous PID-Controlled Line-Following Bot were completed successfully. By integrating an ESP32 microcontroller, a 5-channel reflectance sensor array, and an isolated dual motor drive system, the platform demonstrates reliable path tracking. The addition of anti-windup PID algorithms, gap tolerance filtering, and recovery-pivot logic ensures robust performance on complex track layouts.

9. REFERENCES

1. **ESP32 Technical Reference Manual:** https://www.espressif.com/en/support/documentation/documents
2. **TB6612FNG Driver Datasheet:** https://www.sparkfun.com/datasheets/Robotics/TB6612FNG.pdf
3. **LM2596 DC-DC Converter Specifications:** https://www.ti.com/lit/ds/symlink/lm2596.pdf